

LISTA DE EXERCÍCIOS 2.3

1. Qual é o propósito de chamadas de sistema (*system calls*)?

Resposta: chamadas de sistema permitem que processos em espaço de usuário requisitem serviços do SO.

2. No programa abaixo, explique qual será a saída na LINHA A.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main() {
    pid_t pid;
    pid = fork();
    if (pid == 0) { /* Processo filho */
        value += 15;
        return 0;
    }
    else if (pid > 0) { /* Processo pai */
        wait(NULL);
        printf("PARENT: value = %d",value); /* LINHA A */
        return 0;
    }
}
```

Resposta: O processo filho atualiza a sua cópia do valor, mas lembre-se que isso não é propagado para o processo pai (espaços de memória distintos). Quando o fluxo de controle volta para o processo pai, o valor continua em 5.

3. Incluindo o processo inicial, quantos processos são criados pelo programa mostrado abaixo? Demonstre sua resposta criando uma árvore de processos fictícia.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    /* Crie um processo filho */
```

```

fork();
/* Crie outro processo filho */
fork();
/* e outro */
fork();
return 0;
}

```

Resposta: 8 processos foram criados (rastrear). A chamada `fork()` cria um novo processo, o qual inicia a sua execução no ponto exato onde a chamada `fork()` aconteceu. (ref. <http://stackoverflow.com/questions/1664787/problem-forking-fork-multiple-processes-unix>).

4. Versões originais do SO para dispositivos móveis Apple IOS não ofereciam processamento concorrente. Discuta três complicações que são inseridas para o SO quando este trata processamento concorrente.

Resposta: (1) Um método para *time sharing* deve ser implementado para permitir que cada um de vários processos possua acesso ao sistema. Este método envolve a preempção de processos que não cedem voluntariamente a CPU (pelo uso de uma chamada de sistema, por exemplo) e o kernel ser reentrante (de modo que mais de um processo possa executar código de kernel concorrentemente). (2) Processos e recursos de sistema devem possuir proteções e devem ser protegidos entre si. Qualquer processo deve ser limitado quanto a quantidade de memória que ele pode usar e as operações que ele pode realizar em dispositivos como discos. (3) Deve-se tomar cuidado no kernel para prevenir *deadlocks* entre processos, de modo que processos não permaneçam aguardando entre si por recursos (não-preemptivos) alocados entre si.

5. Quando um processo cria um novo processo usando a chamada de sistema `fork()`, qual dos seguintes estados é compartilhado entre os processos pai e filho?

- a. Stack
- b. Heap
- c. Segmentos de memória compartilhada

Resposta: Somente os segmentos de memória são compartilhados entre o processo pai e o processo filho que sofreu `fork`. Cópias da pilha (stack) e do heap são feitas para o processo filho criado. Mais discussão em <https://stackoverflow.com/questions/4597893/specifically-how-does-fork-handle-dynamically-allocated-memory-from-malloc>

6. Explique as características de IPC utilizando *sockets* e utilizando *pipes*.

Resposta: sockets e pipes são primariamente canais para comunicação entre processos. Sockets são canais para comunicação em rede, definido como um ponto final (endpoint) para comunicação. Consiste na concatenação de informações da camada de rede e da camada de transporte (endereço IP e porta). Por exemplo, o socket 161.25.19.8:1625 refere-se a porta 1625 no host 161.25.19.8. A comunicação é descrita por um par de sockets e pelo protocolo de transporte. Ex. TCP: 161.25.19.8:1625 → 146.86.5.20:80 identifica unicamente essa comunicação na Internet. PIPEs são arquivos especiais no sistema de arquivos. Podem funcionar em modo simplex, half-duplex ou full-duplex dependendo do tipo usado. Via de regra, um PIPE permite que dois processos locais (i.e. compartilhando um mesmo sistema de arquivos) troquem informações.